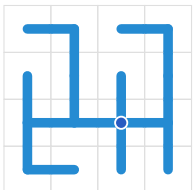
Pipes & Light Up

Giải thuật tìm kiếm cho hai bài toán Logic Puzzle

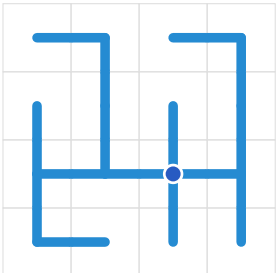


Nhập môn Trí tuệ nhân tạo — BTL1 (HK1 2026–2027)

DFS (mù) + Greedy Best-First (Pipes) · DFS (mù) + A* (Light Up)

[Nhóm — thành viên điền tại đây]

Mục tiêu: một mạng liên thông, không chu trình



- Mọi cổng mở đều gặp cổng tương ứng của ô kề.
- Không cổng nào chìa ra ngoài bàn chơi.
- Tất cả các ô liên thông. Không có chu trình.

Ô là đỉnh,connections là cạnh:

$$\text{G là cây} \implies |E| = n - 1$$

Nguồn xanh chỉ để trực quan hóa nước chảy.

Luật: puzzle-pipes.com. Kiểm tra đích: model.py và search.py.

Phát biểu bài toán tìm kiếm

Input	$n = m^2$ ô, hình dạng cố định, biên thường (không wrap)
Trạng thái	Tiền tố $s = (a_0, \ldots, a_{k-1})$ các định hướng đã gán
Trạng thái đầu	Tiền tố rỗng $()$, biết trước mọi hình dạng
Tác động	Gán một định hướng cho ô k theo thứ tự row-major
Chuyển tiếp	$s' = s + (a_k)$, nhận sau khi qua kiểm tra khả thi
Đích	$k = n$ và bàn hoàn chỉnh đạt mọi luật
Chi phí	1 mỗi phép gán; không giải thuật nào dùng để xếp hạng

Một bước solver có thể tương ứng nhiều lần xoay 90°.
Mọi lời giải hợp lệ đều chấp nhận; không tối ưu số lần xoay.

Mô hình trạng thái cài đặt: Search trong pipes_problem/search.py.

Các successor hợp lệ

Ô kế tiếp có hình dạng cố định; tìm kiếm chọn định hướng của nó.



Ví dụ ô góc trên-trái. Biên trái và trên cấm hướng N và W.

Khớp láng giềng cũng kiểm tra cả cổng không có.
Ứng viên phải qua hết các lớp cắt tỉa dùng chung trước khi được nhận.

Ví dụ minh họa. Lọc định hướng: boundary_domains() và compatible().

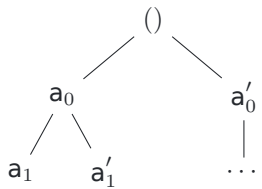
Không gian tìm kiếm

Số xoay phân biệt mỗi ô

Khuỷu, chữ T, đầu cụt	4
Ống thẳng	2
Chữ thập	1

$$U = \prod_{i=0}^{n-1} |R_i| \leq 4^n$$

Cây tiền tố



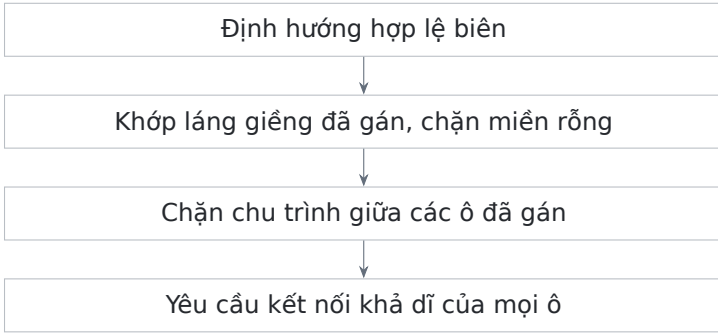
Độ sâu n, hệ số phân nhánh tối đa 4.
Mỗi tiền tố có đúng một cha.

Số cách gán đầy đủ, trước kiểm tra biên.

Ví dụ 4×4 kèm theo: $U = 268{,}435{,}456$.
Đây là cận trên số cách gán, không phải số node đo được.

Đếm từ các xoay phân biệt trong pipes-4x4-261.json.

Cắt tỉa dùng chung



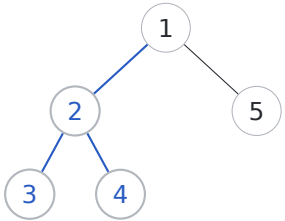
Cả hai giải thuật dùng cùng các kiểm tra và cùng thứ tự ô.

Điều kiện gốc: tổng số cổng $= 2(\mathrm{n} - 1)$.
Kết nối khả dĩ là điều kiện cần, chưa chứng minh tồn tại nghiệm.

Cài đặt cắt tỉa: Search.compatible(). Phân tích đầy đủ: docs/pipes-analysis.md.

Tìm kiếm mù: Depth-First Search

```
VISIT(s):
  if budget exhausted: return LIMIT
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    if FEASIBLE(child):
      r = VISIT(child)
      if r is SOLVED or LIMIT: return r
  return UNSOLVABLE
```



Thứ tự thăm minh họa.
Đi sâu một nhánh rồi quay lui.

Đầy đủ trên cây hữu hạn với đủ tài nguyên và cắt tỉa hợp lệ.
Trường hợp xấu nhất vẫn hàm mũ.

Giả mã rút gọn khớp dfs(). Kiểm tra gốc chạy trước VISIT.

Heuristic của Greedy: số lựa chọn còn lại

$$h(s) = \sum_{i \text{ chưa gán}} \log_2 |D_i(s)|$$

$D_i(s)$ là miền định hướng hợp lệ biên, khớp láng giềng đã gán.

Ứng viên	Kích thước miền còn lại	$h(s)$
A	$(1, 2, 4)$	$0 + 1 + 2 = 3$
B	$(1, 1, 2)$	$0 + 0 + 1 = 1$

Greedy ưu tiên B: giá trị nhỏ hơn nghĩa là ít lựa chọn còn lại.
Thứ tự hòa: tiền tố sâu hơn, rồi tới chèn sớm hơn.

Điểm số bỏ qua ràng buộc giữa các ô chưa gán.
Không admissible, không đảm bảo tối ưu số lần xoay.

Ví dụ miền giả định. Độ ưu tiên thật: (h, -depth, thứ tự chèn) trong greedy().

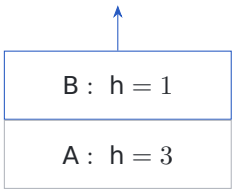
Greedy Best-First search

```
push () into min-priority queue
while queue is not empty:
  if budget exhausted: return LIMIT
  s = pop minimum-priority state
  count expansion
  if GOAL(s): return SOLVED(s)
  for a in rotations of tile |s|:
    child = s + (a)
    D = FEASIBLE(child)
    if D exists:
      push child with (h(child),
                       -|child|, serial)
return UNSOLVABLE
```

Kiểm tra gốc chạy trước khi push. Độ ưu tiên chỉ dùng h.

Chọn từ hàng đợi

mở rộng tiếp



Mọi tiền tố khả thi đều nằm trong hàng đợi.

Bộ nhớ có thể tăng hàm mũ.

Giao thức đo lường

- Cùng 9 map cho cả hai giải thuật.
- Ba map mỗi kích thước, mỗi giải thuật chạy 5 lượt.
- Cùng cắt tỉa, cùng thứ tự ô, cùng giới hạn tài nguyên.



Thời gian không gồm vẽ hình và phát lại. Bộ nhớ là đỉnh cấp phát Python được trace. Chỉ gộp hai lần đo khi kết quả trùng khớp.

Giới hạn: 30 giây tính toán và 200,000 lần mở rộng mỗi lượt.

Chạy batch: benchmark.py. Đo: measure() và worker.py.

Kết quả thực nghiệm

Size	Algorithm	Solved/runs	Time (ms)	Peak (KiB)
4×4	DFS	15/15	3.149	26.984
	Greedy	15/15	3.097	12.359
5×5	DFS	15/15	5.261	53.297
	Greedy	15/15	6.157	19.219
7×7	DFS	15/15	24.469	145.852
	Greedy	15/15	31.695	27.594

Median trên 15 lượt thành công mỗi kích thước và giải thuật.
Mỗi nhóm gồm 3 input phân biệt, 5 lượt bấm giờ.

DFS nhanh hơn ở 5×5 và 2/3 map 7×7; Greedy thắng trên map khó nhất (38.2 so với 40.8 ms) và giữ bộ nhớ traced thấp hơn 4–5 lần.
Input này khá dễ với cắt tỉa hiện có, chưa đủ để kết luận kẻ thắng chung.

Đo ngày 25/10/2026, Linux, Python 3.13. Dữ liệu thô: docs/benchmarks/2026-10-25-pipes.csv.

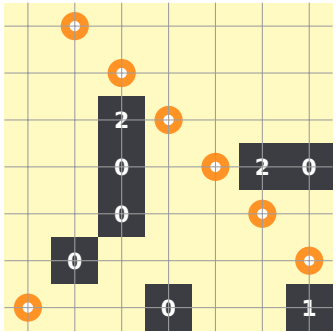
Phạm vi và thí nghiệm tiếp theo

- Thêm map khó hơn và map vô nghiệm đã chứng minh.
- Đo riêng đóng góp từng lớp cắt tỉa (ablation).
- Giữ tách biệt UNSOLVABLE với LIMIT/CANCELLED.
- Thử heuristic khác (MRV) và mở rộng bộ input.

Tài liệu tham khảo

Luật Pipes: `https://www.puzzle-pipes.com/`
Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th ed., chương 3 và 6. `https://aima.cs.berkeley.edu/`

Bài toán 2: Light Up (Akari)



- Đặt bóng đèn vào **ô trắng**; mỗi bóng chiếu 4 hướng tới tường/biên.
- Mọi ô trắng phải được chiếu sáng.
- Hai bóng không được chiếu thẳng vào nhau.
- Tường số k: đúng k bóng kề trực tiếp.

Bàn mẫu thật: `akari-7-easy-70097` do generator sinh,
lời minh họa là nghiệm 7 bóng của nó.

Luật: puzzle-light-up.com. Bàn: generator/generate.py, seed 70097.

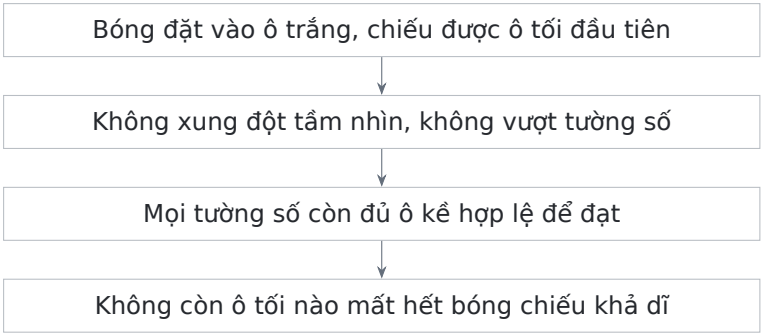
Phát biểu bài toán tìm kiếm

Input	Bàn: `None` (trắng), -1 (tường), $0..4$ (tường số)
Trạng thái	Tập ô có bóng, mã hóa **bitmask** W **bit** (W = số ô trắng)
Trạng thái đầu	Tập rỗng (mask 0)
Tác động	Đặt bóng vào một ô trắng hợp lệ
Chuyển tiếp	Bật bit; loại successor vi phạm luật hoặc mất khả năng đạt
Đích	Mọi ô trắng sáng, mọi tường số đúng, không xung đột
Chi phí	1 mỗi bóng đặt vào (trục tối ưu của A*)

Cả hai giải thuật rẽ nhánh theo **ô tối đầu tiên (row-major)**: chỉ xét bóng hợp lệ chiếu được ô đó.
Mỗi trạng thái một cha $\Rightarrow$ không cần bảng chống lặp phức tạp; so sánh hai giải thuật là công bằng.

Mô hình cài đặt: _View.branch_cell trong light_up_problem/algorithms/search.py.

Cắt tỉa hợp lệ dùng chung



Các kiểm tra chỉ loại trạng thái không thể tới nghiệm $\Rightarrow$ giữ tính đầy đủ.
Đây là lý do DFS “mù” vẫn giải nhanh các bàn vừa và nhỏ.

Cài đặt: _View.feasible() trong algorithms/search.py.

Tìm kiếm mù: Depth-First Search

```
DFS(puzzle, limits):
  push mask 0 onto the stack
  while stack is not empty:
    if budget exhausted: return LIMIT
    state = stack.pop()
    if state in visited: continue
    if GOAL(state): return SOLVED(state)
    cell = FIRST_UNLIT_CELL(state)
    for c in bulbs illuminating cell:
      child = state | bit(c)
      if FEASIBLE(child): push child
  return UNSOLVABLE
```

LIFO stack + tập visited.

Thứ tự rẽ nhánh cố định, không có thông tin heuristic.

Worst case $O(2^W)$ trạng thái.

Lời giải **đầu tiên** tìm được — không tối ưu số bóng.

Giả mã rút gọn khớp depth_first_search(). TIMEOUT/NODE_LIMIT/MEMORY_LIMIT phân biệt UNSOLVABLE.

Heuristic của A*: ba cận dưới, lấy max

$$h(n) = \max\left\{ \left\lceil \frac{D}{G_{\max}} \right\rceil,\ \max_{w} d_w,\ \left\lceil \frac{\sum_w d_w}{H_{\max}} \right\rceil \right\} \qquad f = g + h, \quad g = \#\text{bóng}$$

D: số ô tối còn lại; $G_{\max}$: nhiều ô nhất một bóng hợp lệ chiếu được;
d_w: phần thiếu của tường số w; $H_{\max}$: nhiều kề nhất một bóng thỏa cùng lúc.

Thành phần	Lý do là cận dưới
$\lceil D/G_{\max} \rceil$	Mỗi bóng chỉ chiếu tối đa $G_{\max}$ ô tối
$\max_w d_w$	Tường thiếu d_w cần ít nhất d_w bóng kề
$\lceil \sum_w d_w / H_{\max} \rceil$	Tổng phần thiếu cần đủ nhiều bóng kề

Max của các cận dưới vẫn là cận dưới $\Rightarrow$ h **admissible** $\Rightarrow$ A* trả lời giải **tối ưu số bóng** khi chạy hết.

Cài đặt: _View.lower_bound() trong algorithms/search.py.

A* search

```
ASTAR(puzzle, limits):
  push (f=h(0), g=0, mask 0) into min-heap
  best_g = {0: 0}
  while heap is not empty:
    if budget exhausted: return LIMIT
    pop state with minimum f
    if GOAL(state): return SOLVED(state)
    for c in bulbs illuminating
        FIRST_UNLIT_CELL(state):
      child = state | bit(c)
      if FEASIBLE(child) and
         g+1 < best_g.get(child, inf):
        best_g[child] = g+1
        push (g+1+h(child), g+1, child)
  return UNSOLVABLE
```

Giả mã rút gọn khớp astar(). Chi phí đơn vị nên f không giảm theo đường đi.

Min-heap giữ **toàn bộ frontier**.

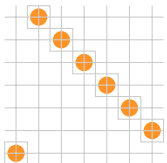
Bảng best_g chống tái khám phá.

Mỗi lần mở rộng tính lại ba cận dưới trên bitmask lớn.

Bộ nhớ có thể tăng hàm mũ — thấy rõ trong số liệu.

Sinh input theo seed, kiểm chứng witness

- Xây witness hoán vị tuần hoàn: mỗi hàng/cột đúng một bóng.
- Đề xuất tường chỉ ở chỗ không phá witness; suy số trên tường từ bóng kề.
- Giữ tường chỉ khi witness vẫn là nghiệm hợp lệ (kiểm bằng chính `is_goal_state` dùng khi chơi).



4 kích thước: 7, 10, 14, 25

3 độ khó: 16%/10.5%/5.5% tường

12 bộ sample phủ đủ tổ hợp

JSON `lightup-puzzle-v1`

Uniqueness chưa kiểm tra

Độ khó gán theo cấu trúc (mật độ tường), chưa hiệu chuẩn thời gian giải.

Cài đặt: generator/generate.py. Witness không lưu vào bàn người chơi.

Demo trực quan trong pygame

- **Chơi thủ công**: click đặt/bỏ bóng; ô sáng tô màu, xung đột viền đỏ, tường số hiển thị đếm hiện tại.
- **Tìm kiếm trực tiếp**: Run / Pause / Step / Stop với bộ đếm expanded / generated / frontier và bộ bóng hiện tại (sự kiện đã lấy mẫu).
- **So sánh DFS và A*** trên cùng bàn: thời gian, node, số bóng hiển thị cạnh nhau.
- **Play solution**: phát lại từng bước đặt bóng của lời giải.

Solver chạy trong process riêng (spawn-safe) nên UI không bị khựng;
giới hạn thời gian/node/bộ nhớ của solver được áp dụng cả trong demo.

Cài đặt: ui/app.py + worker.py.

Giao thức đo

- 12 input sample: 4 kích thước $\times$ 3 độ khó, mỗi giải thuật 3 lượt.
- Giới hạn: 30 s, 500,000 node, 512 MB bộ nhớ ước tính.
- Cùng bàn, cùng giới hạn cho DFS và A*.



Cùng máy, cùng ngày với bảng Pipes (Linux, Python 3.13.14) để hai bài toán so sánh được.
72 lượt dữ liệu thô: `docs/benchmarks/2026-10-25-lightup.csv`.

Chạy: light_up_problem.benchmark. Bằng chứng: docs/benchmarks/2026-10-25-lightup.md.

Kết quả: DFS nhanh và nhẹ, A* tối ưu nhưng đắt

Input	DFS: thời gian / node	A*: thời gian / node	Bóng (DFS → A*)
7 easy	0.026 s / 8	0.273 s / 70	$7 = 7$
7 normal	0.016 s / 9	0.236 s / 153	$8 \to \mathbf{7}$
7 hard	0.027 s / 7	6.77 s / 3164	$6 = 6$
10 easy	0.549 s / 248	3.50 s / 618	$11 \to \mathbf{10}$
10 normal	0.057 s / 13	26.7 s / 5464	$12 \to \mathbf{9}$
10 hard	0.078 s / 12	TIMEOUT 30 s (3621 KiB)	—
14 (cả 3)	0.35–0.42 s / 15–17	TIMEOUT ×3	—
25 (cả 3)	6.7–9.7 s / 33–35	TIMEOUT ×3	—

- DFS giải **12/12** nhờ "ô tối đầu tiên" + cắt tỉa: gần như không quay lui (node ≈ 1.3× số bóng).
- A* hoàn thành **5/12**, chậm hơn 9–250×, nhưng chứng minh tối ưu: 10-normal chỉ cần 9 bóng thay vì 12 (DFS thừa 33%).
- Nới ngân sách A* lên **600 s**: cả 7 input vẫn TIMEOUT (10-hard: 151k node, ∼70 MiB) — TIMEOUT là không kết luận, không phải vô nghiệm.

Median 3 lượt (30 s) + 1 lượt 600 s, đo 25/10/2026, Linux. Dữ liệu: docs/benchmarks/2026-10-25-lightup*.csv.

Hạn chế và hướng phát triển

- Độ khó gán theo mật độ tường, chưa hiệu chuẩn theo thời gian giải.
- Tính duy nhất nghiệm chưa kiểm tra.
- Bộ nhớ đo bằng tracemalloc: đỉnh cấp phát Python, không phải RSS.
- Cận dưới mạnh hơn (matching trên ô tối) có thể kéo A* xa hơn.

Tài liệu tham khảo

Luật Akari: `https://www.puzzle-light-up.com/`
Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th ed., chương 3.
Phân tích đầy đủ: `docs/lightup-analysis.md`.

Tổng kết

- Hai bài toán, bốn giải thuật, cùng một khung phát biểu bài toán tìm kiếm.
- Mọi giải thuật đều demo được từng bước trực quan và đo trên cùng input.
- DFS + cắt tỉa hợp lệ: nhanh và nhẹ trên input vừa; heuristic trả chi phí sắp xếp để đổi lại thứ tự duyệt tốt hơn (Greedy) hoặc tối ưu chi phí (A*).
- Số liệu đầy đủ: `docs/benchmarks/2026-10-25-*.csv`.

Tài liệu tham khảo

`https://www.puzzle-pipes.com/` · `https://www.puzzle-light-up.com/`
Russell & Norvig, Artificial Intelligence: A Modern Approach, 4th ed.
Cài đặt và phân tích: `pipes_problem/`, `light_up_problem/`, `docs/`.